Handwritten Code

```
import java.util.concurrent.atomic.AtomicLong;

public class AAFZaman_Thread implements Runnable {

    private static final AtomicLong safeCounter =
                        new AtomicLong(0);
    private static long unsafeCounter = 0;
    private final long increments;
    private final boolean threadSafe;
    private long instanceCounter = 0;

    public AAFZaman_Thread(long increments, boolean
                                        threadSafe) {
        this.increments = increments;
        this.threadSafe = threadSafe;
    }

@Override
public void run() {
    for (long i=0; i< increment; i++) {
```

```
        if (threadsafe) {
            safeCounter.incrementAndGet();
        } else {
            unsafeCounter++;
        }
        instantCounter++;
    }
}

public long getInstanceCounter() {
    return instanceCounter;
}

public static void main(String[] args)
        throws InterruptedException {
    if (args.length != 3) {
        System.out.println("Usage: java AtiffZaman-
                            Thread <threads>" +
                           "<increments> <true|false>");
        return;
    }
```

```
int numberofThreads = Integer.parseInt(args
long increments = Long.parseLong(args[1]);
boolean threadsafe;
if (args[2].equalsIgnoreCase("true")){
    threadsafe = true;
} else if (args[2].equalsIgnoreCase("false")
    threadsafe = false;
} else {
    System.out.println("Mode must be true or
                        false.
    return;
}
if (numberofThreads <=0 || increments <0){
    System.out.println("Invalid input value.
    return;
}
SafeCounter.set(0);
unsafeCounter =0;
```

```
ARifZaman_Thread[] tasks =
        new ARifZaman_Thread[number of Threads];

Thread[] threads = new Thread[number of Threads];

long expectedCount = (long) number of Threads *
                              increments;

for (int i = 0; i < number of Threads; i++) {
    task[i] = new ARifZaman_Thread(increments,
                                   threadSafe);

    threads[i] = new Thread(task[i]);
    threads[i].start();
}

for (int i = 0; i < number of Threads; i++) {
    threads[i].join();
}

long staticCount = threadSafe ? safeCounter.get()
                              : unsafeCounter;
```

```
long nonStaticTotal = 0;

for (ARFzamon_Thread task : tasks) {
    nonStaticTotal += task.getInstanceCounter();
}

long difference =
        Math.abs(staticCount - nonStaticTotal);

String percentage;

if (nonStaticTotal == 0) {
    percentage = staticCount == 0
        ? "0.00%" : "Undefined"; }
else {
    double percent = 100.0 * difference/nonStatic

    percentage = String.format("%.2f%%", percent
}
```

```
System.out.println("Threads: " + numberOfThreads);
System.out.println("Increments per thread: " +
                   increments);
System.out.println("Mode: " + (threadSafe ? "Thread-Safe"
                   : "Unsafe"));
System.out.println("Expected count: " + expectedCount);
System.out.println("Static count: " + staticCount);
System.out.println("Non-static total: " + nonStaticTotal);
System.out.println("Absolute difference: " +
                   difference);
System.out.println("Percentage difference: " +
                   percentage);
    }
}
```